

Техническая документация: Мост автоматизации Telegram & Email

Полностью очищенная и безопасная версия системы рассылки файлов и уведомлений

1. Архитектура и описание системы

Настоящая система представляет собой независимый автоматизированный шлюз («Мост»), написанный на языке Python 3. Данное решение разработано как альтернатива интеграции через встроенные роботы коммерческих CRM-систем с целью обеспечения полной независимости от тарифных планов и ограничений внешних платформ.

В качестве командного интерфейса используется **выделенная приватная группа в Telegram**. Скрипт в реальном времени отслеживает сообщения в этой группе. При поступлении текста, содержащего номер телефона (и, опционально, адрес электронной почты), система автоматически выполняет парсинг, нормализует контактные данные и осуществляет одновременную отправку целевых файлов (прайс-листов, каталогов, документов) через клиентский аккаунт Telegram (по протоколу MTProto) и почтовый шлюз SMTP.

Важные технические и финансовые предупреждения

- **Ограничения CRM-систем (Финансовый подводный камень):** Доступ к исходящим вебхукам (REST API) и визуальному дизайнеру бизнес-процессов на базовых тарифах многих систем полностью блокируется и требует дорогостоящих подписок на маркетплейсы приложений. Разработанный скрипт полностью автономен, не совершает запросов к внешним интерфейсам и работает бесплатно в любых условиях.
- **Почтовые фильтры и протоколы:** Некоторые почтовые сервисы в современных интерфейсах делают услугу «Пароли для внешних приложений» платной или сложной в настройке. Для обхода этого ограничения в данном скрипте используется прямая авторизация по протоколу SMTP, которая активируется бесплатным тумблером «Доступ к почте по IMAP, POP, SMTP» в настройках безопасности ящика.
- **Специфика прокси-серверов:** При использовании сетевых прокси необходимо жестко разграничивать протоколы. Сетевые порты, выделенные провайдером под HTTP-трафик, не способны пропускать сырой SOCKS5-сигнал. Настройки скрипта приведены в точное соответствие с валидным типом прокси.

2. Безопасность и защита личных данных

В соответствии с высокими стандартами кибербезопасности, из исходного кода скрипта и сопутствующих файлов **полностью удалены все жестко закодированные персональные, корпоративные и сетевые данные.**

Все IP-адреса, логины, пароли, адреса электронной почты, числовые идентификаторы чатов и токены заменены на универсальные текстовые заглушки. Это исключает риски компрометации данных при публикации проекта в публичных или частных репозиториях (например, на GitHub). Для запуска скрипта на целевом сервере необходимо заполнить конфигурационный блок актуальными значениями.

3. Очищенный исходный код системы (`bot.py`)

Ниже представлен универсальный стабильный код управляющего скрипта, готовый к безопасному переносу и публикации:

```
# -*- coding: utf-8 -*-
import os
import re
import smtplib
from email.message import EmailMessage
from telethon import TelegramClient, events

# =====
# БЛОК КОНФИГУРАЦИИ (ЗАПОЛНЯЕТСЯ ПРИ РАЗВЕРТЫВАНИИ)
# =====

# Настройки авторизации Telegram API (получать на my.telegram.org)
API_ID = 1234567 # Замените на ваш числовой API ID
API_HASH = 'ВАШ_API_HASH_СТРОКА' # Замените на ваш API Hash
PHONE_NUMBER = '+799900000000' # Номер телефона вашего аккаунта

# Параметры прокси-сервера (замените на данные вашего прокси)
PROXY_SETTINGS = {
    "proxy_type": "http", # Тип прокси (http или socks5)
    "addr": "ИСПРАВЬТЕ_НА_IP_АДРЕС_ПРОКСИ", # Например: "123.45.67.89"
    "port": 8000, # Порт вашего прокси
    "username": "ЛОГИН_ПРОКСИ",
    "password": "ПАРОЛЬ_ПРОКСИ"
}

# Системный числовой ID приватной группы управления в Telegram
# (Обычно начинается с минуса, например: -1001234567890)
COMMAND_CHAT_ID = -1000000000000

# Настройки исходящей почты
EMAIL_SENDER = 'your_email@domain.com' # Ваш адрес электронной почты
EMAIL_PASSWORD = 'ВАШ_ПАРОЛЬ_ОТ_ПОЧТЫ' # Пароль от ящика или пароль приложения

# Параметры отправляемых материалов
```

```

FILE_TO_SEND = 'price.xlsx' # Имя файла в папке со скриптом (например, price.xlsx
или catalog.pdf)
TEXT_CAPTION = (
    "Здравствуйте! Направляю вам актуальный прайс-лист и сопроводительные материалы.
\n"
    "Вся необходимая информация находится во вложении.\n\n"
    "Буду рад сотрудничеству!"
)

# =====
# ИНИЦИАЛИЗАЦИЯ И ЛОГИКА РАБОТЫ
# =====

# Создание клиента Telegram с туннелированием
client = TelegramClient('automation_bridge_session', API_ID, API_HASH,
proxy=PROXY_SETTINGS)

def send_email_sync(recipient_email):
    """Синхронная функция отправки Email через SMTP с вложением"""
    msg = EmailMessage()
    msg['Subject'] = 'Актуальный прайс-лист и коммерческое предложение'
    msg['From'] = EMAIL_SENDER
    msg['To'] = recipient_email
    msg.set_content(TEXT_CAPTION)

    if os.path.exists(FILE_TO_SEND):
        with open(FILE_TO_SEND, 'rb') as f:
            file_data = f.read()
            file_name_attach = os.path.basename(f.name)
            msg.add_attachment(file_data, maintype='application', subtype='octet-stream',
filename=file_name_attach)
    else:
        return False, f"Файл {FILE_TO_SEND} не найден на сервере"

    try:
        # Использование стандартного защищенного порта 465 для SMTP
        with smtplib.SMTP_SSL('smtp.mail.ru', 465) as server:
            server.login(EMAIL_SENDER, EMAIL_PASSWORD)
            server.send_message(msg)
        return True, "Успешно"
    except Exception as e:
        return False, str(e)

@client.on(events.NewMessage(chats=COMMAND_CHAT_ID))
async def main_handler(event):
    text = event.raw_text

    # Поиск номера телефона и email регулярными выражениями
    phone_match = re.search(r'\+?\d[\d\-\s()]{9,}\d', text)
    email_match = re.search(r'[a-zA-Z0-9_.\+-]+@[a-zA-Z0-9-]+\.[a-zA-Z0-9-.]+', text)

    if not phone_match:
        await event.reply("❌ Ошибка: В сообщении не обнаружен номер телефона
клиента.")

```

```

        return

# Очистка и нормализация номера
raw_phone = phone_match.group(0)
clean_phone = re.sub(r'\D', '', raw_phone)

# Автоматическая коррекция формата номера для СНГ/РФ (+7 -> 8)
if clean_phone.startswith('7') and len(clean_phone) == 11:
    clean_phone = '8' + clean_phone[1:]
elif len(clean_phone) == 10:
    clean_phone = '8' + clean_phone

log_status = f"⌚ Запущена обработка заявки для номера `{clean_phone}`...\n"
report_message = await event.reply(log_status)

# Этап 1. Отправка документа в Telegram клиенту
try:
    await client.send_file(clean_phone, FILE_TO_SEND, caption=TEXT_CAPTION)
    log_status += "✅ Telegram: Документы успешно отправлены клиенту первыми!\n"
except Exception as e:
    log_status += f"❌ Telegram: Не удалось отправить (Возможно, номера нет в мессенджере).\n"

await client.send_message(event.chat_id, log_status, reply_to=report_message.id)

# Этап 2. Отправка документа на Email (при наличии в тексте)
if email_match:
    email_addr = email_match.group(0)
    log_status += f"⌚ Почта: Инициирована отправка письма на `{email_addr}`...\n"

    success, error_details = send_email_sync(email_addr)
    if success:
        log_status += "✅ Почта: Письмо успешно доставлено!\n"
    else:
        log_status += f"❌ Почта: Сбой отправки ({error_details})\n"
else:
    log_status += "ℹ️ Почта: Адрес не указан в запросе, отправка пропущена.\n"

await client.send_message(event.chat_id, log_status, reply_to=report_message.id)

print("Система запущена. Ожидание команд в управляющей группе...")
client.start(phone=PHONE_NUMBER)
client.run_until_disconnected()

```

4. Инструкция по развертыванию на новом сервере (Миграция)

Если вы решите перенести бота на другой VPS или локальный компьютер, строго следуйте этому пошаговому алгоритму:

Шаг 4.1. Подготовка операционной системы (Ubuntu / Linux)

Подключитесь к новому серверу по SSH, обновите репозитории и установите менеджер виртуальных окружений:

```
sudo apt update && sudo apt upgrade -y  
sudo apt install python3-venv python3-pip -y
```

Шаг 4.2. Создание изолированной рабочей среды

```
mkdir my_telegram_bridge && cd my_telegram_bridge  
python3 -m venv venv  
source venv/bin/activate
```

После активации в начале строки терминала появится маркер `(venv)`.

Шаг 4.3. Установка программных зависимостей

```
pip install --upgrade pip  
pip install telethon
```

Шаг 4.4. Размещение файлов конфигурации

- Создайте файл скрипта командой ``nano bot.py``, скопируйте в него очищенный универсальный код из раздела 3, заполните блок настроек своими актуальными данными и сохраните (Ctrl+O, Enter, Ctrl+X).
- Загрузите ваш целевой файл (например, ``price.xlsx``) в эту же папку. Название файла должно строго соответствовать переменной ``FILE_TO_SEND`` в коде.

Шаг 4.5. Первый запуск и прохождение проверок Telegram

```
python3 bot.py
```

Порядок ввода данных:

1. Скрипт запросит код подтверждения, который придет в ваше приложение Telegram. Введите его и нажмите Enter.
2. Если включен Облачный пароль (2FA), введите его в консоль. *Помните, что символы в целях безопасности отображаться не будут — вводите вслепую и нажмите Enter.*
3. После входа нажмите **Ctrl + C** для остановки интерактивного теста. В папке создастся файл сессии ``automation_bridge_session.session``.



Бесперебойный запуск в фоновом режиме (24/7)

Чтобы бот продолжал работать после закрытия терминала, запустите его утилитой фонового вывода:

```
nohup python3 bot.py > server.log 2>&1 &
```

Все системные логи будут записываться в текстовый файл `server.log` в этой же директории.